□ RUST PROGRAMMING

Complete Beginner's Notes

From Zero to Writing Real Rust Programs

Topics Covered

Installation & Setup • Variables & Data Types • Ownership & Borrowing
Functions • Control Flow • Structs & Enums • Pattern Matching
Collections • Error Handling • Traits • Closures & Iterators
Modules & Cargo • Common Patterns • Practice Projects

Chapter 1: Introduction to Rust

What is Rust?

Rust is a systems programming language developed by Mozilla Research, first released in 2015. It is designed to be fast, safe, and concurrent — without needing a garbage collector.

Rust is famous for its unique memory safety model: it prevents bugs like null pointer dereferences, dangling pointers, and data races at compile time, before your program even runs.

Why Learn Rust?

- Memory safety without a garbage collector (GC)
- Blazing fast performance — comparable to C and C++
- Fearless concurrency — the compiler prevents data races
- Rich type system and pattern matching
- Excellent tooling: Cargo (build tool), rustfmt (formatter), Clippy (linter)
- Growing adoption in systems, web assembly, embedded, and cloud

> *Rust has been voted the 'most loved programming language' in the Stack Overflow Developer Survey for 9 consecutive years!*

Installing Rust

The official and recommended way to install Rust is via rustup, the Rust toolchain installer.

On Linux / macOS:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

On Windows:

Download and run the installer from: https://rustup.rs

Verify Installation:

```
rustc --version        # Check Rust compiler version
cargo --version        # Check Cargo version
```

Your First Rust Program

Create a new project using Cargo:

```
cargo new hello_world
cd hello_world
```

```
cargo run
```

The src/main.rs file will look like this:

```
fn main() {
    println!("Hello, world!");
}
```

> ✅*cargo run compiles and runs your project in one step. Use cargo build to only compile.*

Chapter 2: Variables & Data Types

Variables

In Rust, variables are declared with the let keyword. By default, variables are immutable (cannot be changed after assignment).

```
fn main() {
    let x = 5;          // immutable variable
    let mut y = 10;     // mutable variable
    y = 20;             // OK - y is mutable
    // x = 6;           // ERROR - x is immutable
    println!("x = {}, y = {}", x, y);
}
```

> *Making variables immutable by default is a safety feature — it prevents accidental changes.*

Constants

Constants are always immutable, must have a type annotation, and are valid for the entire program runtime.

```
const MAX_POINTS: u32 = 100_000;  // _ is used for readability
const PI: f64 = 3.14159265;
```

Shadowing

You can declare a new variable with the same name as a previous one. This is called shadowing. The new variable 'shadows' the old one.

```
fn main() {
    let x = 5;
    let x = x + 1;       // shadows previous x
    let x = x * 2;       // shadows again
    println!("x = {}", x);  // prints: x = 12
}
```

Data Types

Rust is statically typed — every value has a specific type. The compiler usually infers the type, but you can annotate it explicitly.

Scalar Types (single value):

```
// Integers
let a: i8  = -128;        // signed 8-bit   (-128 to 127)
let b: u8  = 255;         // unsigned 8-bit  (0 to 255)
```

```
let c: i32 = -1_000_000;  // signed 32-bit  (most common)
let d: u64 = 1_000_000;   // unsigned 64-bit
let e: isize = -5;        // platform-dependent (32 or 64 bit)

// Floating point
let f: f32 = 3.14;        // 32-bit float
let g: f64 = 3.14159265;  // 64-bit float (default)

// Boolean
let is_active: bool = true;
let is_done: bool = false;

// Character
let letter: char = 'A';
let emoji: char = '□';   // chars are Unicode (4 bytes)
```

Compound Types:

Tuples — fixed-length collection of values of different types:

```
let tup: (i32, f64, bool) = (500, 6.4, true);
let (x, y, z) = tup;              // destructuring
println!("y = {}", y);             // prints: 6.4
println!("first = {}", tup.0);     // access by index
```

Arrays — fixed-length collection of values of the SAME type:

```
let arr: [i32; 5] = [1, 2, 3, 4, 5];
let repeated = [0; 10];           // [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
println!("first = {}", arr[0]);   // access by index
println!("length = {}", arr.len());
```

> □ *Arrays have a fixed size. If you need a growable list, use a Vec<T> (covered in Chapter 9).*

Chapter 3: Ownership — Rust's Superpower

What is Ownership?

Ownership is Rust's most unique feature. It is a set of rules the compiler checks at compile time to manage memory without a garbage collector.

The three rules of ownership are:

- Each value in Rust has a variable that is its owner.
- There can only be one owner at a time.
- When the owner goes out of scope, the value will be dropped (memory freed).

Move Semantics

When you assign a heap-allocated value to another variable, the original variable is 'moved' and can no longer be used.

```
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;            // s1 is MOVED to s2
    // println!("{}", s1); // ERROR! s1 is no longer valid
    println!("{}", s2);    // OK
}
```

> *Stack types (integers, booleans, chars, tuples of these) are copied, not moved. They implement the Copy trait.*

Clone

If you want to deeply copy heap data, use .clone():

```
let s1 = String::from("hello");
let s2 = s1.clone();        // deep copy
println!("s1={}, s2={}", s1, s2);  // both valid
```

Borrowing & References

Instead of transferring ownership, you can 'borrow' a value using a reference (&). The original owner keeps ownership.

```
fn main() {
    let s1 = String::from("hello");
    let len = calculate_length(&s1);   // borrow s1
    println!("Length of '{}' is {}.", s1, len);  // s1 still valid
}

fn calculate_length(s: &String) -> usize {
```

```
    s.len()      // s is a reference, doesn't own the string
}
```

Mutable References

To modify a borrowed value, use a mutable reference (&mut):

```
fn main() {
    let mut s = String::from("hello");
    change(&mut s);
    println!("{}", s);   // prints: hello, world
}

fn change(s: &mut String) {
    s.push_str(", world");
}
```

Rules for References:

- You can have ANY number of immutable references at the same time.
- OR you can have exactly ONE mutable reference — but not both simultaneously.
- References must always be valid (no dangling references).

> *These rules prevent data races at compile time — no runtime overhead needed!*

The Slice Type

Slices let you reference a contiguous portion of a collection without owning it:

```
let s = String::from("hello world");
let hello = &s[0..5];     // &str slice: 'hello'
let world = &s[6..11];    // &str slice: 'world'

let arr = [1, 2, 3, 4, 5];
let slice = &arr[1..3];   // [2, 3]
```

Chapter 4: Functions

Defining Functions

Functions are defined using the fn keyword. Rust uses snake_case for function names.

```
fn greet(name: &str) {
    println!("Hello, {}!", name);
}

fn add(x: i32, y: i32) -> i32 {
    x + y   // no semicolon = expression = return value
}

fn main() {
    greet("Alice");
    let result = add(3, 5);
    println!("3 + 5 = {}", result);
}
```

> *In Rust, the last expression in a function body is automatically returned. No return keyword needed (but you can use it for early return).*

Statements vs Expressions

This distinction is important in Rust:

- Statements: instructions that perform an action and do NOT return a value. Ending with ;
- Expressions: compute and return a value. Do NOT end with ;

```
fn main() {
    let y = {
        let x = 3;
        x + 1   // expression — this block evaluates to 4
    };
    println!("y = {}", y);  // prints: y = 4
}
```

Multiple Return Values

Rust doesn't have true multiple return values, but you can use tuples:

```
fn min_max(nums: &[i32]) -> (i32, i32) {
    let mut min = nums[0];
    let mut max = nums[0];
    for &n in nums {
        if n < min { min = n; }
```

```
        if n > max { max = n; }
    }
    (min, max)
}

fn main() {
    let nums = [3, 1, 7, 2, 9];
    let (min, max) = min_max(&nums);
    println!("Min: {}, Max: {}", min, max);
}
```

Chapter 5: Control Flow

if / else if / else

```
fn main() {
    let number = 7;

    if number < 0 {
        println!("Negative");
    } else if number == 0 {
        println!("Zero");
    } else {
        println!("Positive");
    }

    // if is an expression - can be used inline
    let label = if number % 2 == 0 { "even" } else { "odd" };
    println!("{} is {}", number, label);
}
```

Loops

loop — infinite loop:

```
let mut count = 0;
let result = loop {
    count += 1;
    if count == 10 {
        break count * 2;   // break returns a value
    }
};
println!("Result: {}", result);  // 20
```

while — loop while condition is true:

```
let mut n = 1;
while n < 100 {
    n *= 2;
}
println!("n = {}", n);  // first power of 2 >= 100
```

for — loop over a collection:

```
// Loop over array
```

```
let arr = [10, 20, 30, 40, 50];
for element in arr {
    println!("value: {}", element);
}

// Loop over a range
for i in 0..5 {
    println!("{}", i);   // 0, 1, 2, 3, 4
}

// Reversed range
for i in (1..=5).rev() {
    println!("{}", i);   // 5, 4, 3, 2, 1
}
```

> ✅*Prefer for loops over while loops when iterating over collections — they're safer and more idiomatic.*

Chapter 6: Structs

Defining and Using Structs

Structs let you group related data together under a meaningful name.

```
// Define a struct
struct User {
    username: String,
    email: String,
    age: u32,
    is_active: bool,
}

fn main() {
    // Create an instance
    let user1 = User {
        username: String::from("alice"),
        email: String::from("alice@example.com"),
        age: 30,
        is_active: true,
    };

    println!("User: {}", user1.username);
    println!("Age: {}", user1.age);
}
```

Struct Update Syntax

```
let user2 = User {
    email: String::from("bob@example.com"),
    username: String::from("bob"),
    ..user1    // use remaining fields from user1
};
```

Methods on Structs (impl)

You can define methods on structs using impl blocks:

```
struct Rectangle {
    width: f64,
    height: f64,
}

impl Rectangle {
```

```
    // Method - takes &self
    fn area(&self) -> f64 {
        self.width * self.height
    }

    fn perimeter(&self) -> f64 {
        2.0 * (self.width + self.height)
    }

    fn is_square(&self) -> bool {
        self.width == self.height
    }

    // Associated function (like a static method) - no &self
    fn new(width: f64, height: f64) -> Rectangle {
        Rectangle { width, height }
    }
}

fn main() {
    let rect = Rectangle::new(10.0, 5.0);
    println!("Area: {}", rect.area());
    println!("Is square: {}", rect.is_square());
}
```

Tuple Structs

```
struct Color(i32, i32, i32);      // RGB
struct Point(f64, f64, f64);      // XYZ

let black = Color(0, 0, 0);
let origin = Point(0.0, 0.0, 0.0);
println!("Red: {}", black.0);
```

Chapter 7: Enums & Pattern Matching

Defining Enums

Enums allow you to define a type by listing its possible variants. Each variant can optionally hold data.

```
// Simple enum
enum Direction {
    North,
    South,
    East,
    West,
}

// Enum with data
enum Shape {
    Circle(f64),              // holds radius
    Rectangle(f64, f64),      // holds width, height
    Triangle(f64, f64, f64),  // holds three sides
}
```

The Option Enum

Rust has no null values. Instead, it uses the Option<T> enum to represent a value that may or may not exist:

```
// Option is built into Rust:
// enum Option<T> {
//     Some(T),
//     None,
// }

fn divide(a: f64, b: f64) -> Option<f64> {
    if b == 0.0 {
        None
    } else {
        Some(a / b)
    }
}

fn main() {
    let result = divide(10.0, 2.0);
    match result {
        Some(val) => println!("Result: {}", val),
        None      => println!("Cannot divide by zero!"),
```

```
    }
}
```

Pattern Matching with match

match is one of Rust's most powerful features. It compares a value against patterns and executes the matching arm.

```
fn describe_number(n: i32) -> &'static str {
    match n {
        0           => "zero",
        1 | 2 | 3   => "small",
        4..=9       => "medium",
        10..=99     => "large",
        _           => "huge or negative",  // _ is wildcard
    }
}
```

if let — Concise Pattern Matching

When you only care about one pattern, if let is simpler than match:

```
let favorite_color: Option<&str> = Some("blue");

if let Some(color) = favorite_color {
    println!("Favorite color is {}!", color);
} else {
    println!("No favorite color.");
}
```

Chapter 8: Error Handling

The Result Enum

For operations that can fail, Rust uses the Result<T, E> enum instead of exceptions:

```
// enum Result<T, E> {
//     Ok(T),
//     Err(E),
// }

use std::fs::File;
use std::io;

fn read_file(path: &str) -> Result<String, io::Error> {
    let mut file = File::open(path)?;  // ? propagates error
    let mut contents = String::new();
    use std::io::Read;
    file.read_to_string(&mut contents)?;
    Ok(contents)
}
```

The ? Operator

The ? operator is a shortcut for handling errors. If the Result is Err, it returns the error from the current function. If Ok, it unwraps the value.

```
fn parse_number(s: &str) -> Result<i32, std::num::ParseIntError> {
    let n = s.trim().parse::<i32>()?;  // ? returns Err if parse fails
    Ok(n * 2)
}
```

Handling Results

```
fn main() {
    let result = parse_number("42");

    // Option 1: match
    match result {
        Ok(val)  => println!("Doubled: {}", val),
        Err(e)   => println!("Error: {}", e),
    }

    // Option 2: unwrap_or (provide default on error)
    let val = parse_number("abc").unwrap_or(0);
```

```
    println!("Value: {}", val);  // 0

    // Option 3: unwrap (panics on error - use only when sure)
    let val = parse_number("10").unwrap();
    println!("Value: {}", val);  // 20
}
```

> ✅*Use ? in functions that return Result. Use match or unwrap_or for graceful error handling. Avoid unwrap() in production code.*

Chapter 9: Common Collections

Vectors (Vec<T>)

Vectors are growable, heap-allocated lists. They are the most commonly used collection in Rust.

```
fn main() {
    // Create a vector
    let mut v: Vec<i32> = Vec::new();
    v.push(10);
    v.push(20);
    v.push(30);

    // Create with values
    let v2 = vec![1, 2, 3, 4, 5];

    // Access elements
    println!("First: {}", v2[0]);          // panics if out of bounds
    println!("Second: {:?}", v2.get(1));   // returns Option<&T>

    // Iterate
    for x in &v2 {
        println!("{}", x);
    }

    // Mutably iterate
    let mut v3 = vec![1, 2, 3];
    for x in &mut v3 {
        *x *= 2;
    }
    println!("{:?}", v3);   // [2, 4, 6]
}
```

HashMaps

HashMaps store key-value pairs. Keys must be unique.

```
use std::collections::HashMap;

fn main() {
    let mut scores: HashMap<String, i32> = HashMap::new();

    scores.insert(String::from("Alice"), 95);
    scores.insert(String::from("Bob"), 87);
```

```
    scores.insert(String::from("Carol"), 91);

    // Access a value
    if let Some(score) = scores.get("Alice") {
        println!("Alice's score: {}", score);
    }

    // Insert only if key doesn't exist
    scores.entry(String::from("Dave")).or_insert(80);

    // Iterate
    for (name, score) in &scores {
        println!("{}: {}", name, score);
    }
}
```

Strings

Rust has two main string types: &str (string slice, borrowed) and String (owned, heap-allocated, growable).

```
// &str - string literal, immutable, fixed
let s1: &str = "hello";

// String - owned, mutable, growable
let mut s2 = String::from("hello");
s2.push(' ');              // add character
s2.push_str("world");    // add string slice

// Concatenation
let s3 = String::from("Hello, ");
let s4 = String::from("world!");
let s5 = s3 + &s4;       // s3 is moved here!

// format! macro (doesn't take ownership)
let s6 = format!("{} - {}", s2, s4);

// Length (in bytes, not characters!)
println!("Length: {}", s2.len());
```

Chapter 10: Traits

What are Traits?

Traits define shared behavior. They are similar to interfaces in other languages. A type implements a trait by providing implementations of its methods.

```
// Define a trait
trait Animal {
    fn name(&self) -> &str;
    fn sound(&self) -> &str;
    // Default method implementation
    fn description(&self) -> String {
        format!("{} says {}", self.name(), self.sound())
    }
}

struct Dog;
struct Cat;

impl Animal for Dog {
    fn name(&self) -> &str { "Dog" }
    fn sound(&self) -> &str { "Woof" }
}

impl Animal for Cat {
    fn name(&self) -> &str { "Cat" }
    fn sound(&self) -> &str { "Meow" }
}

fn main() {
    let dog = Dog;
    let cat = Cat;
    println!("{}", dog.description());  // Dog says Woof
    println!("{}", cat.description());  // Cat says Meow
}
```

Trait Bounds

You can use traits to constrain generic type parameters:

```
// Only accept types that implement Animal
fn make_sound(animal: &impl Animal) {
    println!("{}", animal.description());
}
```

```
// Longer syntax using where clause
fn largest<T>(list: &[T]) -> &T
where
    T: PartialOrd,
{
    let mut largest = &list[0];
    for item in list {
        if item > largest {
            largest = item;
        }
    }
    largest
}
```

Common Built-in Traits

- Display — for printing with {} in println!
- Debug — for printing with {:?} in println!
- Clone — for .clone() method
- Copy — for types copied instead of moved
- PartialEq / Eq — for == and != comparisons
- PartialOrd / Ord — for <, >, <=, >= comparisons
- Iterator — for .iter() chains

```
// Derive common traits automatically
#[derive(Debug, Clone, PartialEq)]
struct Point {
    x: f64,
    y: f64,
}

fn main() {
    let p1 = Point { x: 1.0, y: 2.0 };
    let p2 = p1.clone();
    println!("{:?}", p1);   // Point { x: 1.0, y: 2.0 }
    println!("Equal: {}", p1 == p2);
}
```

Chapter 11: Closures & Iterators

Closures

Closures are anonymous functions that can capture variables from their surrounding scope.

```
fn main() {
    // Basic closure
    let add = |x, y| x + y;
    println!("{}", add(3, 4));   // 7

    // Closure capturing environment
    let offset = 10;
    let add_offset = |x| x + offset;
    println!("{}", add_offset(5));  // 15

    // Multi-line closure
    let double_if_positive = |n: i32| {
        if n > 0 { n * 2 } else { n }
    };
    println!("{}", double_if_positive(-3));  // -3
    println!("{}", double_if_positive(5));   // 10
}
```

Iterators

Iterators allow you to process sequences of items in a lazy, chainable way.

```
fn main() {
    let numbers = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

    // filter - keep only even numbers
    let evens: Vec<i32> = numbers.iter()
        .filter(|&&x| x % 2 == 0)
        .cloned()
        .collect();
    println!("{:?}", evens);  // [2, 4, 6, 8, 10]

    // map - double each number
    let doubled: Vec<i32> = numbers.iter()
        .map(|&x| x * 2)
        .collect();
    println!("{:?}", doubled);

    // filter + map + sum
```

```
    let sum: i32 = numbers.iter()
        .filter(|&&x| x % 2 == 0)
        .map(|&x| x * x)
        .sum();
    println!("Sum of squares of evens: {}", sum);

    // find
    let first_gt_5 = numbers.iter().find(|&&x| x > 5);
    println!("{:?}", first_gt_5);  // Some(6)

    // any and all
    let has_zero = numbers.iter().any(|&x| x == 0);
    let all_positive = numbers.iter().all(|&x| x > 0);
    println!("Has zero: {}, All positive: {}", has_zero, all_positive);
}
```

> ✅*Iterator methods are lazy — they don't execute until you call a consuming method like .collect(), .sum(), .count(), etc.*

Chapter 12: Modules & Cargo

Modules

Modules help organize code into namespaces. You can define modules inline or in separate files.

```
// In main.rs
mod math {
    pub fn add(a: i32, b: i32) -> i32 {
        a + b
    }

    pub fn subtract(a: i32, b: i32) -> i32 {
        a - b
    }
}

fn main() {
    let result = math::add(5, 3);
    println!("{}", result);

    use math::subtract;  // bring into scope
    println!("{}", subtract(10, 4));
}
```

> *Use pub to make items public. Without pub, items are private to their module by default.*

Cargo — Rust's Build Tool

Cargo is Rust's build system and package manager. It handles building, testing, dependencies, and publishing.

Key Cargo Commands:

```
cargo new my_project        # Create new binary project
cargo new my_lib --lib      # Create new library project
cargo build                 # Compile the project
cargo build --release       # Compile with optimizations
cargo run                   # Build and run
cargo test                  # Run tests
cargo check                 # Check without compiling (faster)
cargo doc --open            # Generate and open documentation
cargo add serde             # Add a dependency (crate)
cargo update                # Update dependencies
```

Cargo.toml

The Cargo.toml file is your project manifest — it defines metadata, dependencies, and configuration:

```
[package]
name = "my_project"
version = "0.1.0"
edition = "2021"

[dependencies]
serde = { version = "1.0", features = ["derive"] }
rand = "0.8"
reqwest = { version = "0.11", features = ["json"] }
```

Using External Crates

```
// After adding rand to Cargo.toml:
use rand::Rng;

fn main() {
    let mut rng = rand::thread_rng();
    let random_number: i32 = rng.gen_range(1..=100);
    println!("Random number: {}", random_number);
}
```

Chapter 13: Lifetimes (Intro)

What are Lifetimes?

Lifetimes are Rust's way of ensuring that references are always valid. Every reference has a lifetime — the scope for which that reference is valid. Most of the time, the Rust compiler infers lifetimes automatically.

You only need to write lifetime annotations when the compiler can't figure it out — usually when a function takes multiple references and returns one.

```
// Without lifetime annotation - this would be a compile error
// fn longest(x: &str, y: &str) -> &str { ... }

// With lifetime annotation
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}

fn main() {
    let s1 = String::from("long string");
    let result;
    {
        let s2 = String::from("xy");
        result = longest(s1.as_str(), s2.as_str());
        println!("Longest: {}", result);
    }
}
```

> *The lifetime annotation 'a means: 'the returned reference lives at least as long as the shorter of x and y'. It doesn't change how long things live — it just tells the compiler relationships between lifetimes.*

Lifetime in Structs

```
// Struct that holds a reference needs lifetime annotation
struct Important<'a> {
    message: &'a str,
}

fn main() {
    let novel = String::from("Call me Ishmael. Some years ago...");
    let first_sentence;
```

```
    {
        let i = novel.split('.').next().expect("Could not find a '.'");
        first_sentence = Important { message: i };
        println!("{}", first_sentence.message);
    }
}
```

Chapter 14: Writing Tests

Unit Tests

Rust has built-in support for testing. Tests are annotated with #[test] and live in a tests module marked with #[cfg(test)].

```
fn add(x: i32, y: i32) -> i32 {
    x + y
}

fn divide(a: f64, b: f64) -> Option<f64> {
    if b == 0.0 { None } else { Some(a / b) }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_add() {
        assert_eq!(add(2, 3), 5);
        assert_eq!(add(-1, 1), 0);
        assert_ne!(add(2, 2), 5);
    }

    #[test]
    fn test_divide() {
        assert_eq!(divide(10.0, 2.0), Some(5.0));
        assert_eq!(divide(5.0, 0.0), None);
    }

    #[test]
    #[should_panic(expected = "divide by zero")]
    fn test_panics() {
        panic!("divide by zero");
    }
}
```

```
cargo test            # run all tests
cargo test test_add   # run only tests matching 'test_add'
```

Chapter 15: Common Patterns & Tips

Useful String Methods

```
let s = String::from("  Hello, World!  ");
println!("{}", s.trim());             // "Hello, World!"
println!("{}", s.to_lowercase());     // "  hello, world!  "
println!("{}", s.to_uppercase());     // "  HELLO, WORLD!  "
println!("{}", s.contains("World")); // true
println!("{}", s.replace("World", "Rust")); // "  Hello, Rust!  "

let parts: Vec<&str> = "a,b,c".split(',').collect();
println!("{:?}", parts);  // ["a", "b", "c"]

let joined = parts.join(" - ");
println!("{}", joined);  // "a - b - c"
```

Reading User Input

```
use std::io;

fn main() {
    println!("Enter your name:");
    let mut input = String::new();
    io::stdin().read_line(&mut input).expect("Failed to read");
    let name = input.trim();
    println!("Hello, {}!", name);
}
```

Converting Types

```
// String to number
let num: i32 = "42".parse().unwrap();
let num: i32 = "42".parse().unwrap_or(0);

// Number to String
let s = 42.to_string();
let s = format!("{}", 42);

// i32 to f64
let i: i32 = 5;
let f: f64 = i as f64;
```

```
// f64 to i32 (truncates decimal)
let f: f64 = 3.99;
let i: i32 = f as i32;  // i = 3
```

Useful Vector Methods

```
let mut v = vec![3, 1, 4, 1, 5, 9, 2, 6];
v.sort();                           // [1, 1, 2, 3, 4, 5, 6, 9]
v.dedup();                          // remove consecutive duplicates
v.reverse();                        // reverse in place
println!("len: {}", v.len());
println!("is empty: {}", v.is_empty());
v.retain(|&x| x > 3);              // keep only > 3
v.extend([10, 11, 12]);             // add multiple elements
let total: i32 = v.iter().sum();
let max = v.iter().max().unwrap();
let min = v.iter().min().unwrap();
```

Chapter 16: Practice Projects

Project 1: Number Guessing Game

```
// Add to Cargo.toml: rand = "0.8"
use rand::Rng;
use std::cmp::Ordering;
use std::io;

fn main() {
    let secret = rand::thread_rng().gen_range(1..=100);
    println!("Guess the number (1-100)!");

    loop {
        let mut guess = String::new();
        io::stdin().read_line(&mut guess).expect("Read failed");
        let guess: u32 = match guess.trim().parse() {
            Ok(num) => num,
            Err(_)  => { println!("Enter a valid number!"); continue; }
        };
        match guess.cmp(&secret) {
            Ordering::Less    => println!("Too small!"),
            Ordering::Greater => println!("Too big!"),
            Ordering::Equal   => {
                println!("You win!");
                break;
            }
        }
    }
}
```

Project 2: Simple Calculator

```
use std::io;

fn calculate(a: f64, op: &str, b: f64) -> Option<f64> {
    match op {
        "+" => Some(a + b),
        "-" => Some(a - b),
        "*" => Some(a * b),
        "/" => if b != 0.0 { Some(a / b) } else { None },
        _   => None,
    }
}
```

```
fn main() {
    println!("Simple Calculator (e.g., 5 + 3)");
    let mut input = String::new();
    io::stdin().read_line(&mut input).unwrap();
    let parts: Vec<&str> = input.trim().split_whitespace().collect();
    if parts.len() != 3 {
        println!("Invalid input"); return;
    }
    let a: f64 = parts[0].parse().unwrap_or(0.0);
    let op = parts[1];
    let b: f64 = parts[2].parse().unwrap_or(0.0);
    match calculate(a, op, b) {
        Some(result) => println!("= {}", result),
        None => println!("Error: invalid operation or division by
zero"),
    }
}
```

Quick Reference Cheatsheet

Keywords

```
let      - declare variable (immutable)
let mut  - declare mutable variable
const    - constant (always immutable)
fn       - function
struct   - struct type
enum     - enum type
impl     - implementation block
trait    - trait definition
use      - bring item into scope
mod      - module
pub      - make item public
match    - pattern matching
if let   - single-pattern match
loop     - infinite loop
while    - conditional loop
for      - iterating loop
break    - exit loop
continue - skip to next iteration
return   - early return from function
self     - instance reference in method
Self     - type of current impl block
&        - reference (borrow)
&mut     - mutable reference
*        - dereference
->       - return type of function
=>       - match arm
::       - path separator
?        - propagate error
..       - exclusive range (0..5 = 0,1,2,3,4)
..=      - inclusive range (0..=5 = 0,1,2,3,4,5)
```

Common Macros

```
println!("text {}", var)   - print with newline
print!("text {}", var)     - print without newline
eprintln!("error")         - print to stderr
format!("text {}", var)    - create a String
vec![1, 2, 3]              - create a Vec
panic!("message")          - crash with message
assert!(condition)         - panic if false
```

```
assert_eq!(a, b)            - panic if a != b
assert_ne!(a, b)            - panic if a == b
dbg!(variable)              - debug print and return value
todo!()                     - placeholder, panics if reached
unimplemented!()            - like todo!
```

Formatting Specifiers

```
{}      - Display (normal output)
{:?}    - Debug (for debugging)
{:#?}   - Pretty-printed Debug
{:5}    - Width of 5 (right-aligned)
{:<5}   - Width of 5, left-aligned
{:05}   - Zero-padded to width 5
{:.2}   - 2 decimal places
{:8.2}  - Width 8, 2 decimal places
{:b}    - Binary
{:x}    - Hexadecimal (lowercase)
{:X}    - Hexadecimal (uppercase)
{:o}    - Octal
{:e}    - Scientific notation
```

Memory Model Summary

- Stack: fast, fixed-size, LIFO. Holds: integers, booleans, chars, tuples of those, references.
- Heap: flexible size, holds Strings, Vecs, and other dynamically sized data. Accessed via a pointer on the stack.
- Ownership: each value has one owner. When owner drops, value is freed.
- Borrowing: use & to lend a value temporarily without transferring ownership.
- Lifetimes: ensure borrowed values don't outlive their owners.